V ⊗ W
V ⊕ W
A ↓ B
∏
∑_n

Components

X	object
f: X → Y	arrow
A	category
F: A → B ∈ B^A	functor i.e. a path
B^A = { A → B }	functor category
A(X, Y) ∈ Set	hom set
A(-, Y): A → Set ∈ Set^A	hom functor
φ: F ⇒ G	natural transformation
Nat(F, G) = { F ⇒ G }	

f x g: XxY → X'xY'
f ∧ g: Z → XxY
p ∧ q: XxY → [X|Y]

f ⊔ g: X⊔Y → X'⊔Y'
i ∨ j: X⊔Y → Z

==========

======

=== Universal properties ====

Isomorphism F: A → B

	B		A
adjoint	{ FX → Y }	<=>	{ X → -FY }
			inverse
functor	F: A → B isomorphic		-F: B → A isomorphic
	identity		**identity**
base	1Y: F(-F)Y → Y = Y → Y		1X: X → (-F)FX = X → X
base map	F* ∈ F[_→ -FY]		-F* ∈ -F[FX→_]

Equivalent functor F: A → B

	B		A
adjoint	{ FX → Y }	<=>	{ X → GY }

functor	F: A → B		G: B → A
	isomorphism		**isomorphism**
base	εY: FGY → Y		ηX: X → GFX
base map	εY * F		F * ηX

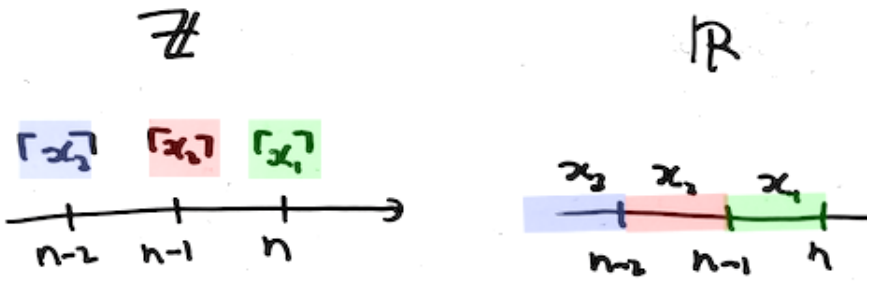
Reflection F: A → A' ⊆ A

	A' ⊆ A		A
adjoint	{ FX → X' }	<=>	{ X' → X }
	reflector		**inclusion**
functor	F: A → A' '		K: A' → A faithful (+ full if A' is a full subcategory)
base	εX': FKX' → X' = FX' → X' epic (+ isomorphic if A' full)		ηX: X → KFX = X → FX
base map	εX' * F ∈ F[_→X']→X'		F * ηX ∈ X→K[FX→_] = X→[FX→_]

I: Z → R injection
Integers Z as a subset of reals R
i.e. **injection I: Z → R**

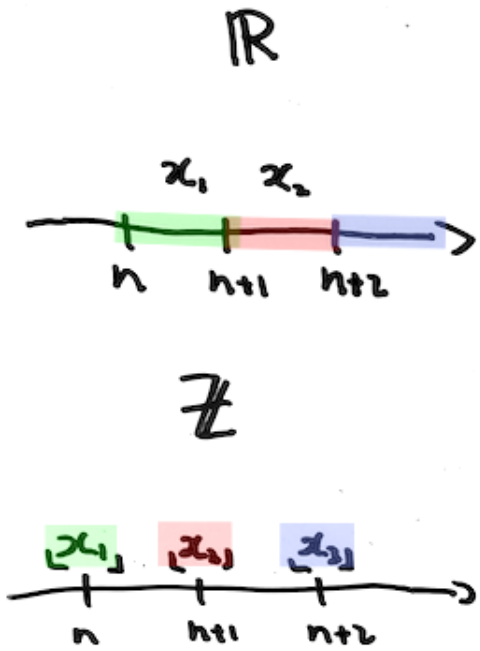
Left: **ceil** ⌈_⌉: R → Z

	Z		**R**
set	⌈x⌉ <= n	<=>	x <= n
functor	⌈_⌉ : R→Z full, faithful		I: Z→R full, faithful
base	1n: ⌈n⌉ <= n = n isomorphic i.e. identity		ηx: x <= I⌈x⌉ isomorphic
base map	1n * ⌈_⌉ ∈ (⌈_ <= n⌉ <= n) = (⌈_ <= n⌉ = n)		I * ηx ∈ (x <= I[⌈x⌉ <=_]) = (x <= [⌈x⌉ <=_])



Right: floor ⌊x⌋: R → Z

	R		**Z**
set	n <= x	<=>	n <= ⌊x⌋
	Inclusion		**Floor**
functor	I: R→Z full, faithful		⌊x⌋ : R → Z full, faithful
base	εx: I ⌊x⌋ <= x isomorphic		1n: n<= I ⌊n⌋ = n isomorphic
base map	εx * I		⌊⌋ * 1n
	∈ (I[_ <= ⌊x⌋] <= x) = (_ <= x)		∈ (n <= ⌊n<=_⌋) = (n <= ⌊_⌋)



Initial/terminal object

1-map [⊤]: A → ●
i.e. everything maps to ●

Right: initial object □ ∈ **A**

	A	<=>	1
adjoint	{ ⊤ → X }		{ ● → ● }
	= { δX: ⊤ → X }		= { 1● }
	Initial object		**1-map**
functor	[⊤]: 1 → A		1: A → 1
	[⊤](1●) = 1⊤		1 (f) = 1●
	i.e. constant functor		i.e. constant functor
	full, faithful		full
	initial arrow		**identity**
base	δX: ⊤ → X		1●
base map	δX * [⊤]		* 1●
	∈ [⊤][● → ●] → X		∈ ● → ●[⊤ → _]
	i.e. single map		i.e. single map
	δX * [⊤] (1●) = 1⊤ ∘ δX		●(δX) = 1●
	= δX		

Duality
Idea: for any map f: X x Y → Z, there is a 1-1 correspondence

x ↦ (y ↦ f(x, y)) <=> **y ↦ (x ↦ f(x, y))**

	Vect	**Vect^op**
adjoint	{ h: V → DW }	{ h: D'V → W }
		= { W → DV }
opposite		
	Bilinear functionals S = { V x W → K }	
	h(v) = { f(v, ⋅): W → K | f ∈ S }	h(w) = { f(⋅,w): V → K | f ∈ S }

	Linear functionals	**Linear functionals (dual)**
functor	D: Vect^op → Vect	D': Vect → Vect^op
	object	object
	DV = { V → K }	D'V = DV
	linear functional V → K	
	arrow u: V → W	arrow u: V → W
	Du^op: DW → DV	D'u: DW → DV
	Du^op(f) = fu: V → K	D'u = Du^op
	for f: W → K	

	Evaluation		**Evaluation**
base	e: V → DD'V		e': D'DV → V
	e(v): D'V → K		e'(v) = e(v)
	e(v)(f) = f(v)		note: input

is V since e' is opposite
for f: V → K

==========
======

=== Galois Extension ====

Galois Extension
A special case of adjoints with posets

Theorem: Galois Extension
Let
P, Q preorders
L: P → Q^op
R: Q → P^op

If

LX <= Y <=> X >= RY

then L, R are adjoints.
Hence,

base
X >= RLX **greatest lower bound** of X. i.e.
if X >= RY then RLX >= RY

LRY <= Y **least upper bound** of Y. i.e.
if LX <= Y, then LX <= LRY

==========
proof
- greatest lower bound
Simply by isomorphism of base map.
If X >= RY, then there is a unique LX <= Y such that
X >= RLX >= RY

QED
==========

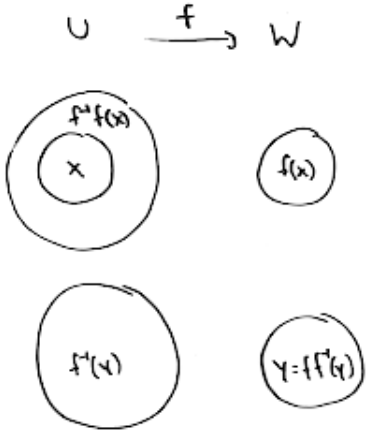
<u>* Examples</u>
<u>Image/preimage</u>
Let
U, W set
f: U → W

Then f, f^-1

	2^U		**2^W**
	X ⊆ -f(Y)	<=>	f(X) ⊆ Y
base	X ⊆ -f(f(X))		f(-f(Y)) = Y

GL bound
$-f(f(X)) \subseteq -f(Y)$
for any $X \subseteq f^{-1}(Y)$

LU bound
$f(X) \subseteq f(-f(Y))$
for any $f(X) \subseteq Y$



Boolean logic
Let
U, W sets

X OR Y

	2^U		**(2^U, 2^U)**
	$X \cup Y \subseteq Z$	<=>	$(X, Y) \subseteq \Delta Z$
base	$Z \cup Z = Z$		$(X, Y) \subseteq \Delta(X \cup Y)$

boolean: $t \in X$ OR Y if $t \in X$ OR $t \in Y$

X AND Y

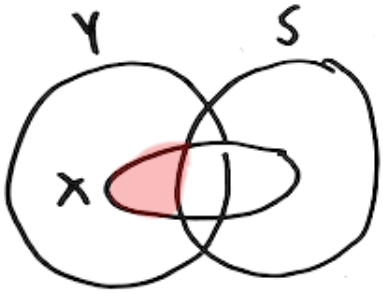
	2^U		**(2^U, 2^U)**
	$Z \subseteq X \cap Y$	<=>	$\Delta Z \subseteq (X, Y)$
base	$Z = Z \cap Z$		$\Delta(X \cap Y) \subseteq (X, Y)$

boolean: $z \in (X$ AND $Y)$ if $z \in X$ AND $z \in Y$

NOT S + S = 1

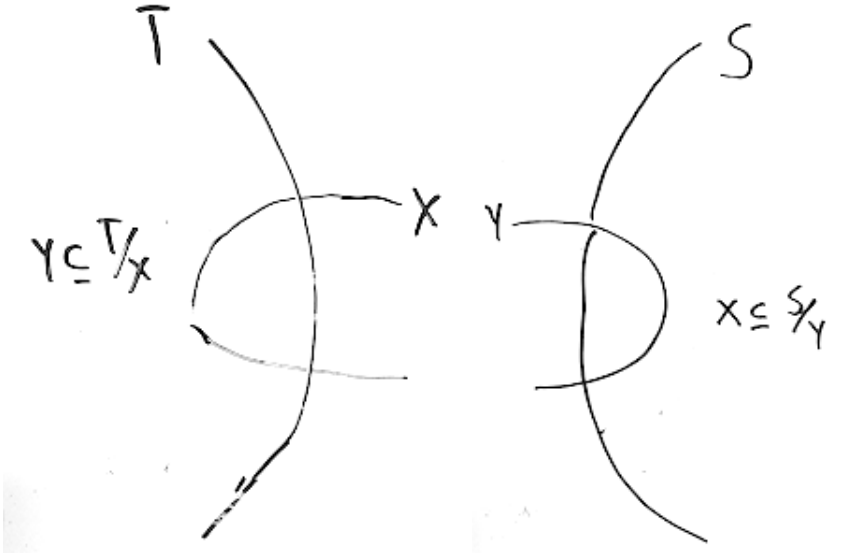
	2^U		**2^U**
	$X / S \subseteq Y$	<=>	$X \subseteq S \cup Y$
base	$Y / S \subseteq Y$		$X \subseteq S \cup X/S$

boolean: 1 = S + NOT S



NOT (NOT S) = S
Let X ⊆ S, Y ⊆ T

	2^U		**2^U**
	T / X ⊆^op Y = Y ⊆ T / X	<=>	X ⊆ S / Y
base	Y ⊆ T / (S / Y)		X ⊆ S / (T / X)
	boolean: Y = NOT (NOT Y)		**boolean**: X = NOT (NOT X)



==========

======

=== Limits ====

0 arrow
f ∘ _ = _ ∘ f = _

<u>* Discrete limits: 0, 1, 2, 3, ...</u>

<u>Empty functor []: 0 → A</u>
i.e. no objects nor morphisms

Lim [] = terminal ⊥ ∈ **A** **Colim [] = initial** ⊤ ∈ A

limit	Lim [] = ⊥	Colim [] = ⊤
	i.e. constant limit	
base	-	-
base map	-* ∈ Conj [_→⊥] ⫤	*- ∈ Conj ⫧[⊤ → _]
	i.e. each X ∈ A has a single "no cone" X⫤	i.e. each X ∈ A has a single "no cone" ⫧X
	{ X → Lim []} <=> ∅	∅
<=>	{ ⊤ → X }	
	= { X → ⊥ } i.e. no cones	

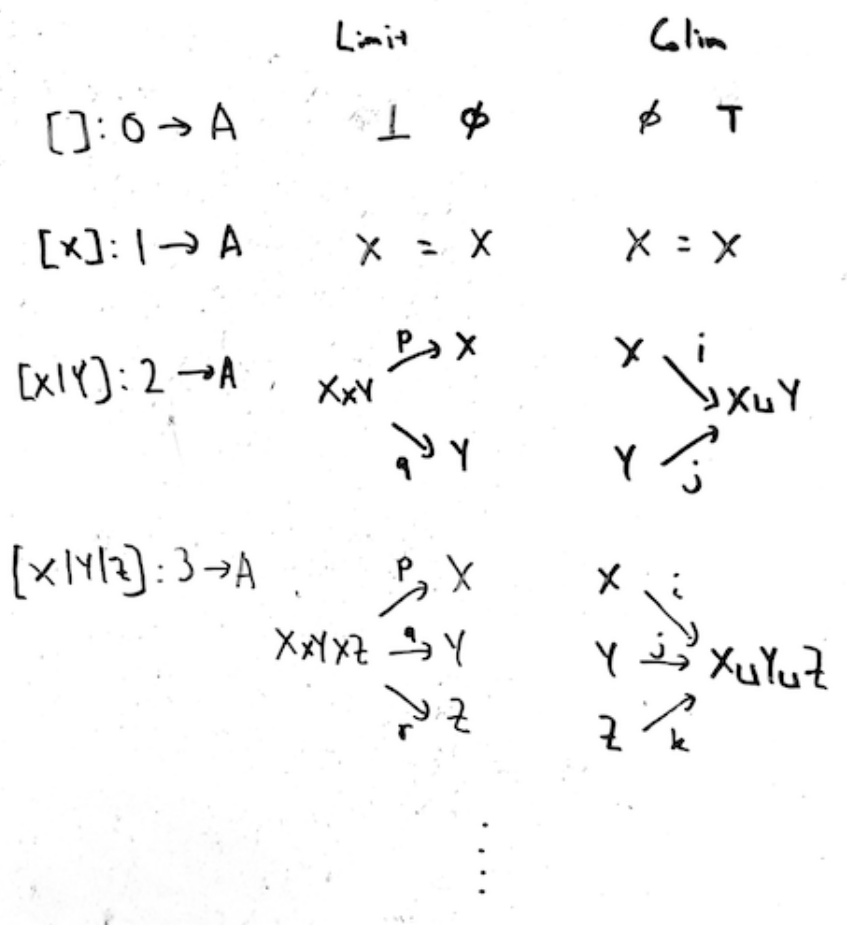
1-functor 1 → A
Discrete map [1]: 1 → A
[X] 1 = X

	Lim [X] = X	**Colim [X] = X**
limit	Lim [X] = X	Colim [X] = X
	identity	**identity**
base	1X = ⊲[X]	1X = [X]⊳
	isomorphic	isomorphic
base map	*1X ∈ [_ → X]⊲[X]	1X* ∈ [X]⊳[X→_]
	{ Z→X } <=> { Z⊲[X] }	{ X⊳Z } <=> { X→Z }

2-functor 2 → A
Discrete map [X|Y]: 2 → A
[X|Y] = { X, Y }
[X|Y] 1 = X
[X|Y] 2 = Y

	Lim [X|Y] = product XxY	**Colim [X|Y] = coprdocut X ⊔ Y**
limit	Lim [X|Y] = X x Y	Colim [X|Y] = X ⊔ Y
	projections	**injections**
base	p ∧ q= ⊲ [X|Y]	i ∨ j = [X|Y]⊳
base map	(p ∧ q)⊲* ∈ Conj [Z→XxY]⊲[X|Y]	*⊳(i ∨ j) ∈ Conj [X|Y] ⊳ [X⊔Y→_]
	{ Z → XxY } <=> { Z⊲ [X|Y] }	{ [X|Y] ⊳ Z } <=> { X ⊔ Y → Z }

Finite products



*** Arrow limits**

[f]: {↓}→ A
f: X → Y

	Lim	**Colim**
limit	Lim [f] = X	Colim [f] = Y
base	1X f ∘ 1X = f	1Y 1Y ∘ f = f
conjugate	*1X◁ ∈ Conj [_ →X]◁[f]	*▷1Y ∈ Conj [f]▷[Y→_]
	{ Z → X } <=> { Z ◁[f] }	{ [f] ▷ Z } <=> { Z → X }

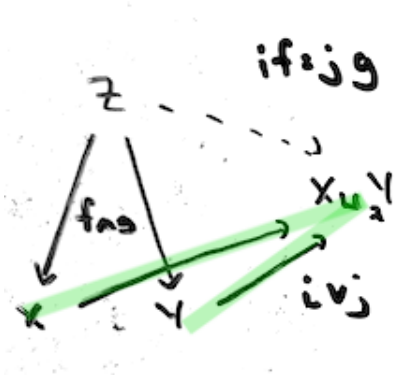
[f|_g]: {↓ | ↓}→ A
f, g: X→Y

	Limit	**Colimit**
limit	Lim [f|g] = Eq f,g	Colim [f|g] = Coeq f,g
base	eq: Eq f,g → X f ∘ eq = g ∘ eq	coeq: Y → Coeq f,g coeq ∘ f = coeq ∘ g
conjugate	eq◁* ∈ Conj [_→ Eq f,g] ◁ [f|g]	*▷coeq ∈ Conj [f|g] ▷ [Coeq f,g

→ _]

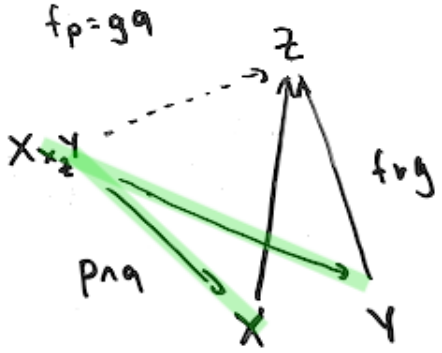
[f ∧ g]: {↓ ∧ ↓}→ A
f: Z → X, g: Z → Y

	Limit	**Colimit (pushout)**
limit	Lim [f∧g] = Z	Colim [f∧g] = X ⊔ Y
base	1Z f ∘ 1Z = f g ∘ 1Z = g	i ∨ j: [X|Y] → X ⊔ Y (i ∨ j) ∘ (f ∧ g) = if = jg
conjugate	1Z⊲* ∈ Conj [_ → Z]⊲[f∧g]	*⊳(i ∨ j) ∈ Conj [f∧g] ⊳ [X ⊔ Y → _]

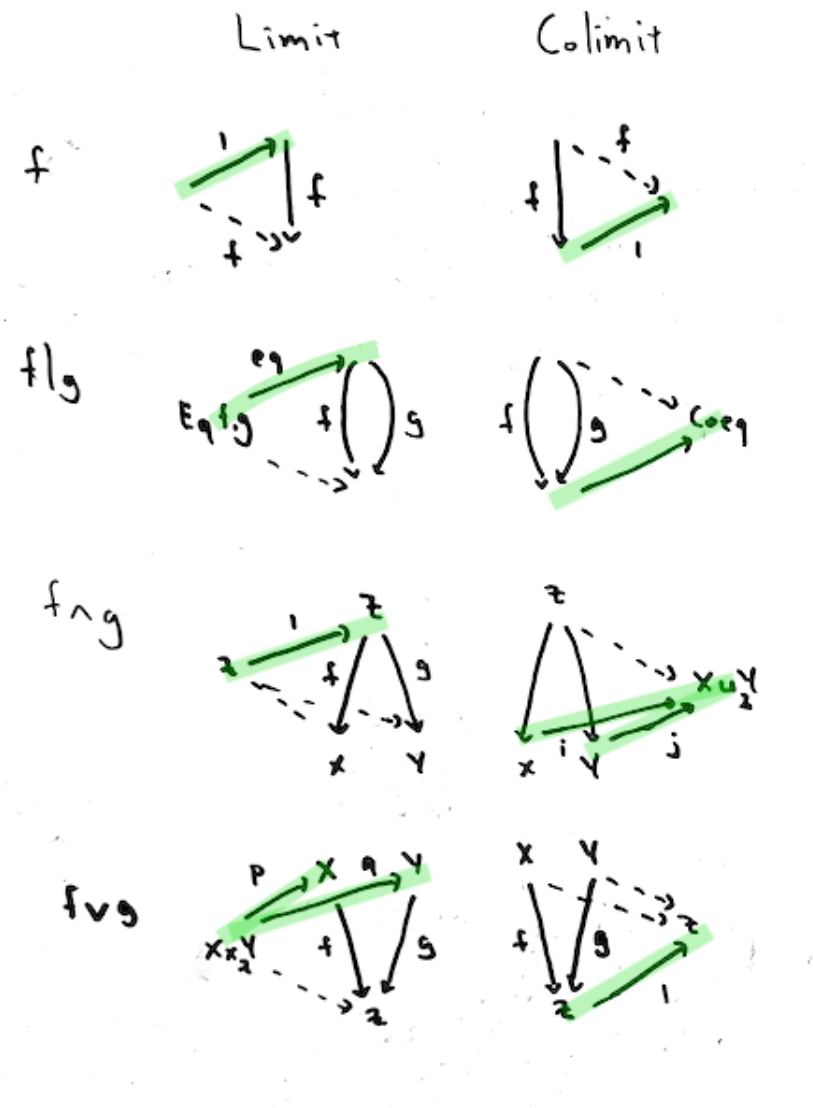


f ∨ g: {↓ ∨ ↓} → A
f: X → Z, g: Y → Z

	Limit (pullback)	**Colimit**
limit	Lim [f ∨ g] = X x Y	Colim [f ∨ g] = Z
base	p ∧ q: XxY → [X|Y] (f ∨ g) ∘ (p ∧ q) = fp = gq	1Z 1Z ∘ f = f 1Z ∘ g = g
conjugate	(p ∧ q)⊲* ∈ Conj [_ → XxY] ⊲ [f ∨ g]	*⊳1Z ∈ Conj [f ∨ g] ⊳ [Z→_]



Limits



System of equations

	Expressions	**arrows**
vertical vector	(xi) = [x1, x2, ...] = x1 AND x2 AND x3 ...	∏j Xj
	scalar	
	[a1,a2, ...] x = [a1x, a2x, ...]	aj: X → Yj a: X → ∏j Yj
	dot product	
	[a1, a2, ...] · [x1, x2, ...] = a1x1 + a2x2 + ...	aj: Xj → Y a: ∏j Xj → Y
	matrix product	
	(aij) [x1,x 2, ...] = [(a1j) · x, (a2j) · x, ...] = [a11x1 + a12x2 + ..., a21x1 + a22x2 ~ ..., ...	aij: Xj→ Yi a: ∏j Xj → ∏i Yi

horizontal vector	(xj) = { x1, x2, ..}	⊔i Xi
	= x1 OR x2 OR x3 ...	
	scalar	
	{ a1, a2, ...} x	ai: X → Yi
	= { a1x, a2x, ...}	a: X → ⊔i Yi
	dot	
	{ a1, a2, ...} ・ {x1, x2, ...}	aj: Xj → Y
	= { a1x1, a2x2, ... }	a: ⊔j Xj → Y
	matrix	
	(aij) {x1, x2, ...}	aij: Xi→ Yj
	= { (a1j) ・ x, (a2j) ・ x, ...]	a: ⊔i Xi → ⊔j Yj
	= [{ a11x1, a12x2, ... },	
	{ a21x1, a22x2, ...},	
	...	
equalizer	Ax = Bx	f ∘ eq = g ∘ eq
	xA = xB	coeq ∘ f = coeq ∘ g

==========

==== Scratch ====

Correspondence

base map	**line**
F * η ∈ Y→F[X→_]	Fx + η
limit	**least upper bound**
*⊲F: [_ → Lim F] ⊲F	x <= inf sup F
∏j Fj → ∏k Gk	**system** of equations i.e. matrix gk = f1 + f2 + ...
equalizer	**solution** to system of equations

Free objects

Free: Set → Vect U: Vect → Set

U * ηX ∈ Conj X→U[Free(X)→_]
ηX: X → U(Free(X))
ηX (x) = x
injective (Free faithful)

εW * Free ∈ Conj Free[_→UW]→W
εW: Free(UW)→W
surjective (U faithful)

Free: Graph → Cat U: Cat → Graph

U * ηG ∈ Conj G→U[Free(G)→_]
εA * Free ∈ Conj Free[_→UA]→A

Quotient

/N: Grp → */N I: */N → Grp

qG: G → I(G/N)
I * qG ∈ Conj G→I[G/N→_]

terminal element ■
■⊳ = 1■
■⊳* ∈ Conj _→■

δX: X → ■
Lim * δX ∈ Conj X → Lim [X ⊲■]
= X → ■

product

p|q⊳: XxY⊲ X|Y
p|q⊳* ∈ Conj [_→XxY] ⊲ X|Y

δZ: Z→ZxZ
Lim * δZ ∈ Conj Z→Lim[Z ⊲ _]
= Z→[ZxZ→_x_]

⊲i|j: X|Y⊳ X⊔Y
*⊲i|j ∈ Conj X|Y ⊳ [X⊔Y→_]

δZ: Z⊔Z→Z
δZ * Lim ∈ Conj Lim[_⊳Z]→Z
= [_⊔_→Z⊔Z]→Z

==========

======

--- [OLD] Examples ---

Isomorphism

Left	B ⊇	FX → F-FY = Y	X ↦ FX
Right	A ⊇	X → -FY	Y ↦ -FY

Poset

Left	R ⊇	n <= ⌊x⌋ <= x	n ↦ n
Right	Z	n <= ⌊x⌋	x ↦ ⌊x⌋

Left	Z ⊇ ⌈x⌉ <= n	x ↦ ⌈x⌉
Right	R ⊇ x <= ⌈x⌉ <= n	n ↦ n

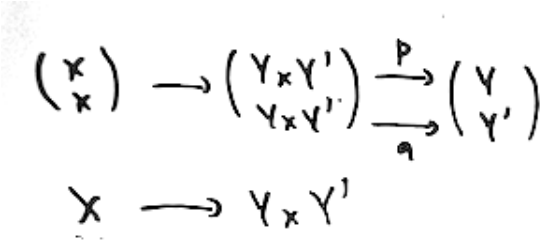
Initial/terminal

Left	A ⊇ X → 0	* ↦ 0
Right	1 ⊇ * → *	X ↦ *
Left	1 ⊇ * → *	X ↦ *
Right	A ⊇ X → 0	* ↦ 0

Product/coproduct

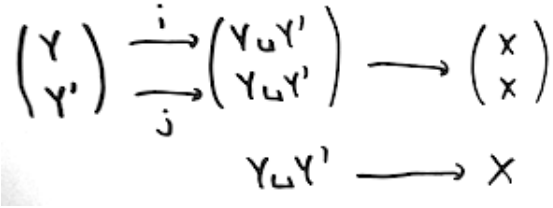
projection

Left	AxA ⊇ (X,X) → (YxY', YxY') → (Y,Y')	X ↦ (X, X)
Right	B ⊇ X → YxY'	(Y, Y') ↦ YxY'



Left	B ⊇ Y⊔Y' → X	(Y, Y') ↦ Y⊔Y'
Right	AxA ⊇ (Y,Y') → (Y⊔Y',Y⊔Y') → (X, X)	X ↦ (X, X)

injection



Exponent

evaluation

Left	A ⊇ XxY → Z^Y x Y → Z	X ↦ XxY
Right	A ⊇ X → Z^Y	Z ↦ Z^Y

Free

interpretation

Left	Vect ⊇ Free(X) → Free(UW) → W	X ↦ Free(X)
Right	Set ⊇ X → UW	W ↦ UW

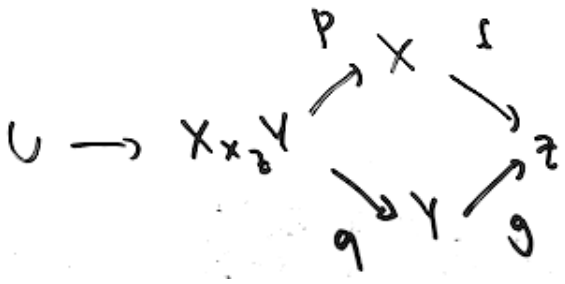
Limit

Left	B^A ⊇ **Δ**X ⇒ **Δ**Lim F ⇒ F	X ↦ **Δ**X
Right	B ⊇ X → Lim F	F ↦ Lim F
Left	B ⊇ Colim F → X	F ↦ Colim F
Right	B^A ⊇ F ⇒ **Δ**Colim F ⇒ **Δ**X	X ↦ **Δ**X

Pullback/pushout

Left →← ⊇ **Δ**U ⇒ **Δ**X xZ Y ⇒ (X→Z←Y) X ↦ **Δ**X = 1X

Right A ⊇ U → X xZ Y (X→Z←Y) ↦ X xZ Y



Left A ⊇ X ⊔Z Y → U (X←Z→Y) ↦ X ⊔Z Y

Right ←→ ⊇ (X←Z→Y) ⇒ **Δ**X ⊔Z Y ⇒ **Δ**U X ↦ **Δ**X = 1X

Isomorphism F: A → B

	B		A
adjoint	{ FX → Y }	<=>	{ X → -FY }
			inverse
functor	F: A → B		-F: B → A
	identity		**identity**
units	1Y: F(-F)Y → Y = Y → Y		1X: X → (-F)FX = X → X

Equivalent functor F: A → B

	B		A
adjoint	{ FX → Y }	<=>	{ X → GY }
functor	F: A → B		G: B → A
	isomorphism		**isomorphism**
unit	εY: FGY → Y		ηX: X → GFX

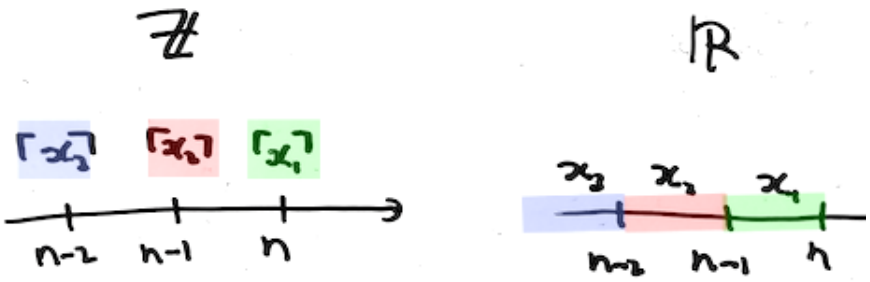
Reflection F: A → A' ⊆ A

	A' ⊆ A		A
adjoint	{FX → X'}	<=>	{X → X'} = {X → KX'}
	reflector		**inclusion**
functor	F: A → A' '		K: A' → A
	epic (isomorphic if A' full)		
unit	εX': FX' → X' = FKX' → X'		ηX: X → FX = X → KFX

I: Z → R **injection**
Integers Z as a subset of reals R
i.e. **injection I: Z → R**

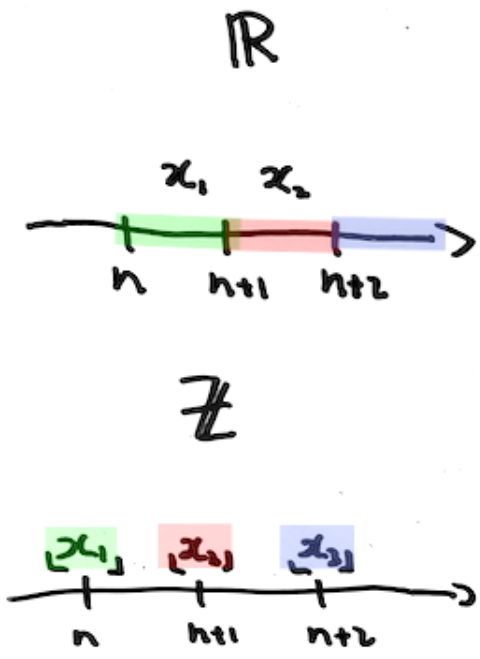
Left adjoint
ceil ⌈x⌉: R → Z

Z		**R**
⌈x⌉ <= n	<=>	x <= n



Right adjoint
floor ⌊x⌋: R → Z

R		**Z**
n <= x	<=>	n <= ⌊x⌋



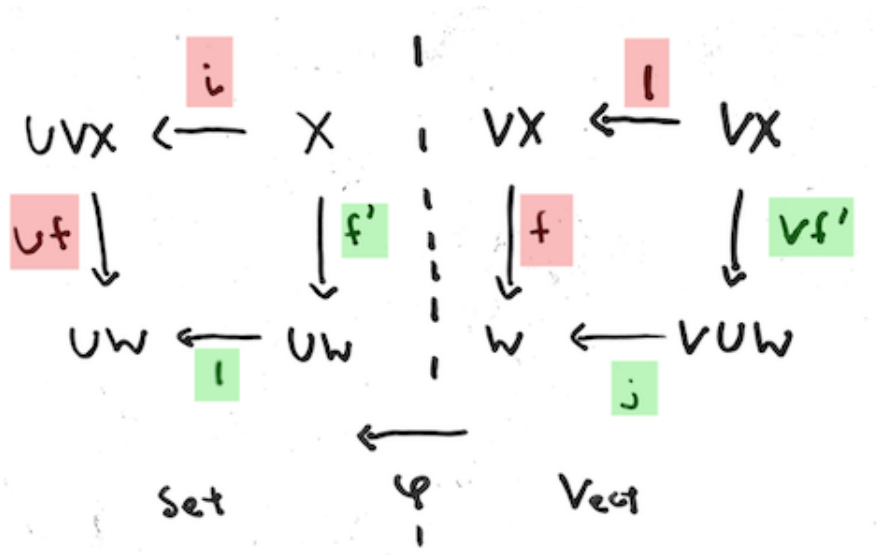
U: Vect → Set

	Vect		Set
adjoint	{ VX → W }	<=>	{ X → UW }
	Free - faithful		**Forget** - faithful
functor	V: Set → Vect		U: Vect → Set
	VX = free vector space generated by X		
	Vf: = extension of f: X → X by linearity		
	interpretation - epic		**inclusion** - monic
unit	j: VUW → W		i:X → UVX
	j = φ^-1(1UW)		i = φ(1VX)
	j(∑ wi) = ∑ wi		i(x) = x

Faithful/full

iX: X → UVX **monic** <=> V **faithful**
not full e.g. f: VX → VX where
f(a) = a+a cannot be induced by X → X

jW: VUW → W **epic** <=> U **faithful**
not full e.g. f: UR → UR where
f(0) = 1 cannot be linear



Diagonal **Δ**: A → A x A

Δ:A → A x A

ΔX = (X, X) object map

Δf = (f, f) morphism map

- **Left Δ**: product YxY'

	AxA		**A**
adjoint	{**Δ**X → (Y, Y')}	<=>	{X → YxY'}
	diagonal - faithful		**product** - faithful
functor	**Δ**: A → AxA		x: AxA → A

Δf = (f, f) x(f, f') = fxf' (as bifunctor)

	product - epi	**diagonal** - monic
units	(pY, qY'): **Δ**YxY' → (Y, Y')	δX: X → XxX
	e.g. if A = Set	
	p(y, y') = y q(y, y') = y'	δX(x) = (x,x)

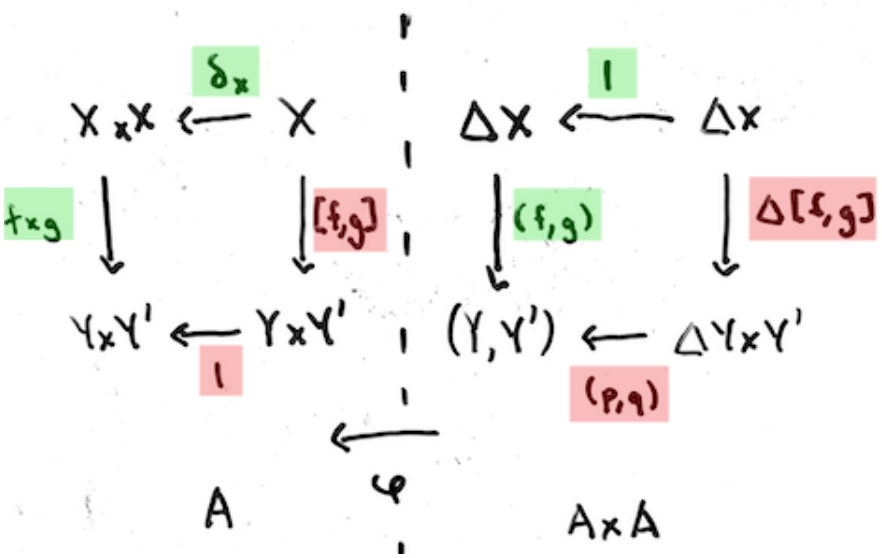
<u>full/faithful</u>

δX: X → XxX **monic** <=> **Δ** faithful

not full e.g f: **Δ**R → **Δ**R where f(1,1) = (0, 1) cannot be a single R → R

(p, q): **Δ**YxY' → (Y,Y) **epic** <=> YxY' faithful

not full e.g. f:RxR → RxR where f(1,0) = (1,0) f(1,1) = (0,0) cannot be of the form f = (g, g')

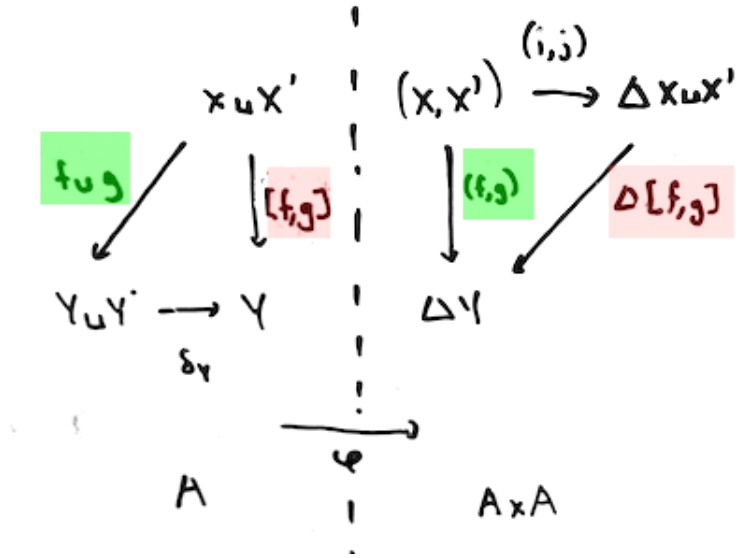


- **Right Δ**: coproduct X ⊔ X'

	A		AxA
φ:	{X⊔X' → Y}	<=>	{(X, X') → **Δ**Y}

	coproduct - faithful	**diagonal** - faithful
functor	⊔: (A, A) → A	**Δ**: A → AxA
	⊔(f, f') = f ⊔ f'	**Δ**f = (f, f)

	collapse - epic	**injection** - monic
units	δY: Y⊔Y → Y	(iX, jX'): (X, X') → **Δ**X⊔X'
	e.g. if A = Set	
	δ(y) = y	i(x) = x j(x') = x'



Diagonal **Δ**: A → *

Δ: A → *

ΔX = * object map

Δf = 1 morphism map

Single object O functor

O: * → A

O* = O ∈ A object map

O1 = 1O morphism map

- **Left**: terminal object Z

	*		A
adjoint	{**Δ**X → *}	<=>	{X → O*}
	= **{ 1*** }		= **{** φ**1*** **}**
			i.e. there is exactly one function **X** → **O**
			=> O is **terminal**
	Diagonal		**terminal** - full, faithful
functor	**Δ**: A → *		O: * → A
			O(1*) = 1O
	identity - isomorphism		**terminal arrow**
unit	1*: * → *		η**X: X → O**

full/faithful

1*: * → * **isomorphic** <=> O **full, faithful**

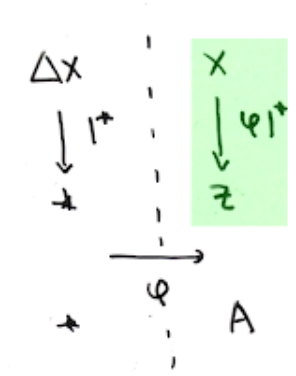
since 1O: O → O is the only arrow

(terminal)

ηX: X → O - <=> **Δ** not faithful

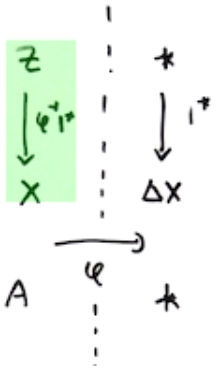
not full 1*: **Δ**X → **Δ**X'

does NOT imply f:X → X' exists



- **Right**: initial object O

	A		*****
adjoint	{O* → X}	<=>	{* → **Δ**X}
	= **{ -φ(1*) }**		= **{ 1* }**
			i.e. there is exactly one
function O → X			
			=> O is **initial**
	Initial - full, faithful		**Diagonal**
functor	O: * → A		**Δ**: A → *
	O(1*) = 1O		
unit	**initial arrow**		**identity** - isomorphic
	εX: O → X		1* = φ(O*)
			1*: * → *



- t**erminal object** <T0, **Δ**>: * → A

	*		A
	{ *X → * }	<=>	{ X → T0 }
	Diagonal		**terminal**
functor	*: A → *		T0: * → A
	f = 1		T0(1*) = 1T0
	identity - isomorphic		**terminal**
units	1*: * → *		ηX: X → T0
1*: * → * **isomorphic**		<=>	T0 **full, faithful**
			since 1T0: T0 → T0 is the only
arrow (terminal)			

ηX: X → T0 - <=> * not faithful
not full 1*: ΔX → ΔX'

does NOT imply f:X → X' exists